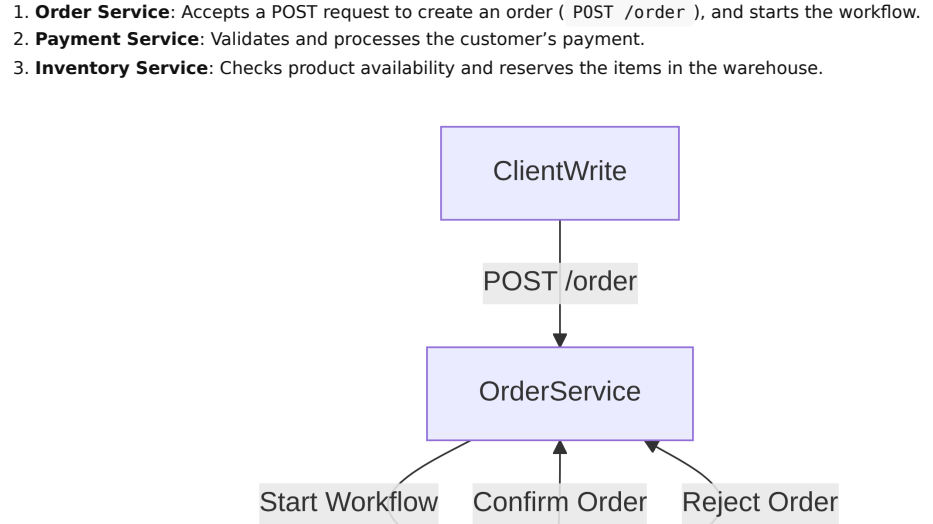
Labs

Lab 1: Orchestration-based Saga

In this exercise, three services collaborate to implement a **Saga Pattern** using Orkes Conductor. The services are:

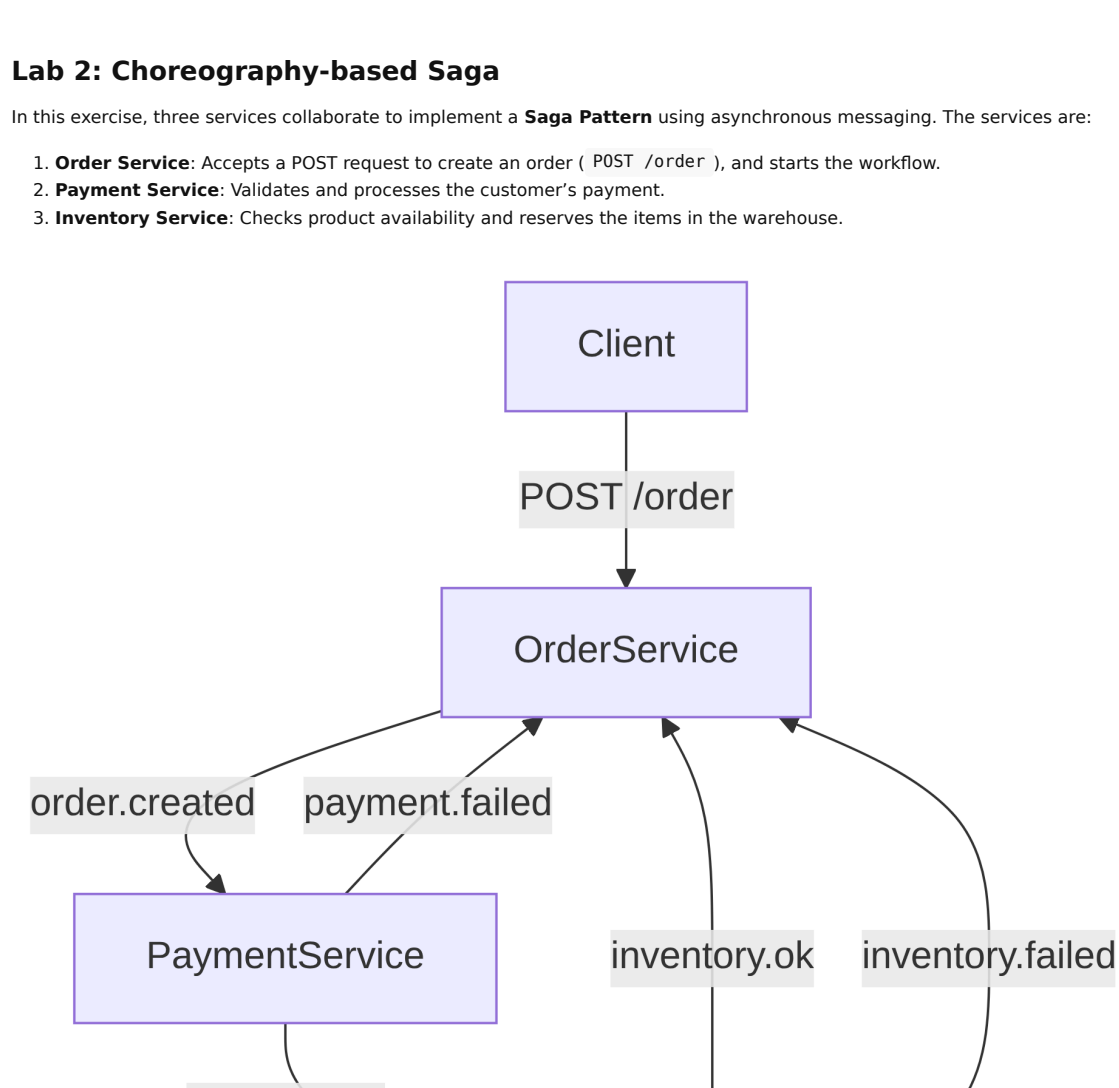
1. **Order Service**: Accepts a POST request to create an order (`POST /order`), and starts the workflow.
2. **Payment Service**: Validates and processes the customer's payment.
3. **Inventory Service**: Checks product availability and reserves the items in the warehouse.


ClientWrite
POST /order
OrderService
Start Workflow
Confirm Order
Reject Order
OrkesConductor
Check Payment
Payment Success
Payment Failed
Check Inventory
Inventory Success
Inventory Failed
PaymentService
InventoryService

Lab 2: Choreography-based Saga

In this exercise, three services collaborate to implement a **Saga Pattern** using asynchronous messaging. The services are:

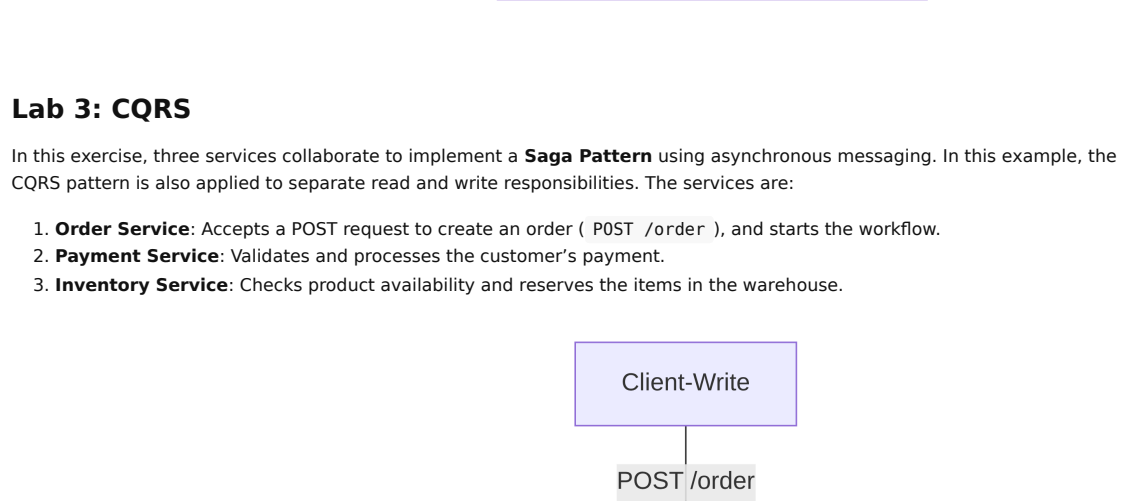
1. **Order Service**: Accepts a POST request to create an order (`POST /order`), and starts the workflow.
2. **Payment Service**: Validates and processes the customer's payment.
3. **Inventory Service**: Checks product availability and reserves the items in the warehouse.


Client
POST /order
OrderService
order.created
payment.failed
PaymentService
inventory.ok
inventory.failed
payment.ok
InventoryService

Lab 3: CQRS

In this exercise, three services collaborate to implement a **Saga Pattern** using asynchronous messaging. In this example, the CQRS pattern is also applied to separate read and write responsibilities. The services are:

1. **Order Service**: Accepts a POST request to create an order (`POST /order`), and starts the workflow.
2. **Payment Service**: Validates and processes the customer's payment.
3. **Inventory Service**: Checks product availability and reserves the items in the warehouse.


Client-Write
POST /order
Client-Read
OrderService-Write
GET /order
order.confirmed
order.created
payment.failed
OrderService-Read
PaymentService
inventory.ok
inventory.failed
payment.ok
InventoryService

Questions

1. What challenges arise in maintaining data consistency when using the one database per service pattern in microservices?
2. What is the Saga pattern, and why is it essential for managing distributed transactions? Describe the choreography and orchestration approaches.
3. What does the CAP theorem state, and how does it affect the design of distributed systems?
4. What role do compensating transactions play in the Saga pattern, and why must they be idempotent?
5. What is the two-phase commit protocol, and how does it ensure atomicity in distributed transactions?
6. What are the performance and scalability trade-offs of using two-phase commit in a distributed environment?
7. What is Conductor, and how does it help manage distributed workflows?
8. What challenges drive the adoption of the CQRS pattern in large-scale applications?
9. How does the CQRS pattern enable independent scaling and performance optimization for read and write operations?
10. How can a CQRS-based system align with either the CP or AP model under the CAP theorem?